

Market Risk Project Report

ESILV — Market Risk (2025–2026)

Students (TD IF2)

Alexis HANNA GERGUIS
Samy DAGHASTANI

Teaching Assistant: Cassandre LEFEVRE
Module Coordinator: Matthieu GARCIN

December 30, 2025

Contents

A Question A - Non-parametric Value-at-Risk	2
A.a Non-parametric VaR estimation (2015–2016)	2
A.b Out-of-sample validation (2017–2018)	6
Conclusion	7
B Question B - Expected Shortfall	8
Objective and definition	8
Computation (consistent with Question A)	8
Results and comparison with VaR	9
Conclusion	9
C Question C - EVT (Pickands estimator) and EVT-based VaR	10
C.a Pickands tail index (right tail vs left tail)	10
C.b EVT-based Value-at-Risk (Pickands, loss tail)	11
Conclusion	12
D Question D - Price impact (Bouchaud’s model)	14
Objective	14
Estimation of variables	17
Results and Interpretation	18
Conclusion	18
E Question E - Haar wavelets, multiresolution correlations, and Hurst exponent	19
E.a Multiresolution correlations with Haar wavelets	19
E.b Hurst exponent and volatility scaling	21
Conclusion	24
General conclusion	25

Introduction

This project was carried out in the context of the *Market Risk* course at ESILV (academic year 2025–2026). The point is not only to “compute a VaR”, but to apply the course methods on real data, justify our choices, and then look at the results with a critical eye (including the moments where reality does not behave nicely).

We provide two complementary supports:

- **This report**, which is meant to be self-contained and readable. We focus on the main steps, the key assumptions, and the final outputs. We also include the most relevant code blocks (as requested), but we try to keep them targeted, so the report stays pleasant to read.
- **Jupyter notebook**, which contains the full pipeline, intermediate checks, and additional plots. If someone wants to reproduce everything line by line (or see where a specific plot comes from), the notebook is the safest place to look.

We will occasionally refer to the notebook when we skip an intermediate step or a technical detail. It is not a way to avoid explanations: it is just our way to keep the report focused and avoid turning it into a long code dump.

Project repository. The complete project is publicly available on GitHub. The repository contains the full Jupyter notebook, all datasets, figures, and auxiliary files required to reproduce the results presented in this report:

https://github.com/AlexisAHG/MarketRisk_Project

The project is structured into five questions (A to E), following the official statement:

- **A:** Non-parametric Value-at-Risk on daily Natixis returns, using a biweight kernel density estimator, and an out-of-sample backtest.
- **B:** Expected Shortfall associated with the VaR from question A, and a practical comparison between both risk measures.
- **C:** Extreme Value Theory: Pickands tail index estimation (both tails), then EVT-based VaR for several confidence levels under an i.i.d. assumption.
- **D:** Calibration of Bouchaud’s price impact model on the TD4 dataset, with a discussion about parameter meaning and model specification.
- **E:** Haar wavelets: multiresolution correlations between FX rates (Epps effect), then Hurst exponent estimation and volatility scaling.

Throughout the report, unless explicitly stated otherwise, we use the standard convention for daily VaR/ES reporting with 95% as the baseline confidence level, and additional levels when it is useful. The exact levels are always easy to change in the code, and we keep the code consistent with the notebook.

A Question A - Non-parametric Value-at-Risk

A.a Non-parametric VaR estimation (2015–2016)

In this question, we estimate a one-day Value-at-Risk (VaR) on Natixis daily returns using a fully non-parametric approach based on a **biweight kernel** density estimator (KDE). We follow the standard definition of VaR as a left-tail quantile.¹ This non-parametric approach is consistent with the course framework and allows us to avoid restrictive distributional assumptions, which is particularly relevant for financial returns that often exhibit asymmetry and heavy tails.

Data, returns, and sample split

We start from daily closing prices and compute simple daily returns:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

We then split the sample into two periods: 2015–2016 for the estimation step, and 2017–2018 for the out-of-sample validation (the idea is: learn a VaR threshold on one period, and see how it behaves later). This estimation/validation split follows the standard backtesting logic introduced in the lectures and ensures that the VaR is evaluated on data that were not used for its calibration.

Code (data loading)

```
data["Date"] = pd.to_datetime(data["Date"], dayfirst=True)
data["Price"] = data["Price"].str.replace(",", ".").astype(float)

data = data.sort_values("Date")
```

Code (returns computation)

```
data["Return"] = data["Price"].pct_change()
data = data.dropna()
```

Code (train/test split)

```
data_2015_2016 = data[
    (data["Date"] >= "2015-01-01") & (data["Date"] <= "2016-12-31")
]

data_2017_2018 = data[
    (data["Date"] >= "2017-01-01") & (data["Date"] <= "2018-12-31")
]
```

Kernel choice and KDE setup

The statement requires the **biweight kernel**:

$$K(u) = \frac{15}{16}(1 - u^2)^2 \mathbf{1}_{|u| \leq 1}.$$

We use the KDE as a practical smoothing step: it keeps the approach non-parametric while making the tail behaviour easier to inspect (especially on the left side, where VaR lives). From a risk-measure perspective, smoothing the empirical distribution also makes the extraction of tail quantiles more stable, while remaining fully non-parametric.

¹Market Risk lecture slides, M. Garcin, p. 68.

Formulas used (KDE)

² Using the returns sample $\{r_1, \dots, r_n\}$ and a bandwidth $h > 0$, the kernel density estimator is

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - r_i}{h}\right).$$

In our implementation, $K(\cdot)$ is the biweight kernel above, and h is selected using a rule-of-thumb (as in the notebook).

Code (biweight kernel)

```
def biweight_kernel(u):  
    return (15/16) * (1 - u**2)**2 * (abs(u) <= 1)
```

Code (bandwidth + KDE function)

```
#we focus on the returns observed over the estimation period (2015-2016)  
returns = data_2015_2016["Return"].values  
n = len(returns)  
  
# Bandwidth selection using a rule-of-thumb formula  
h = 1.06 * np.std(returns) * n ** (-1/5)  
  
def kde_biweight(x, data, h):  
    u = (x - data) / h  
    return np.sum(biweight_kernel(u)) / (len(data) * h)
```

Density plot (sanity check)

Before extracting a quantile, we plot the KDE density. This plot is mainly a sanity check: we want to see if the distribution looks roughly symmetric or clearly skewed, and we want a first visual feel for the left tail (because VaR is a tail object).

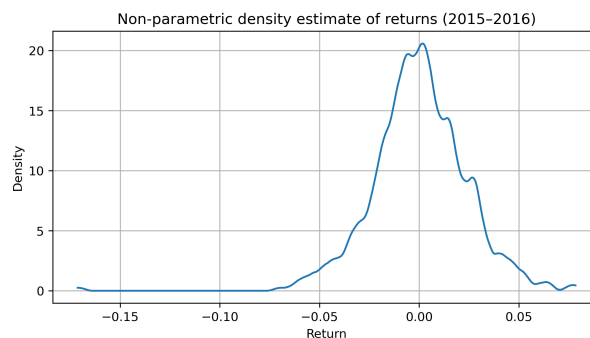


Figure 1: Non-parametric density estimate of returns (2015–2016)

What we take from this plot. This plot confirms that the return distribution departs from a Gaussian shape and that the left tail deserves particular attention. It therefore provides a qualitative consistency check before extracting a quantitative tail risk measure such as the Value-at-Risk.

²Market Risk lecture slides, M. Garcin, p. 71.

Code (density plot)

```
# Before computing the VaR we plot the estimated density to visually check
# the shape of the return distribution especially the left tail.
x_grid = np.linspace(returns.min(), returns.max(), 1000)

kde_vals = np.array([kde_biweight(x, returns, h) for x in x_grid])

plt.figure(figsize=(8, 4))
plt.plot(x_grid, kde_vals)
plt.title("Non-parametric density estimate of returns (2015-2016)")
plt.xlabel("Return")
plt.ylabel("Density")
plt.grid(True)
plt.show()
```

VaR computation (KDE-based quantile)

VaR at level α is defined as the α -quantile of the return distribution:

$$\text{VaR}_\alpha = F^{-1}(\alpha).$$

In this report (and in the notebook), VaR is computed directly on returns: it is a left-tail quantile and therefore comes out as a negative threshold.

Formulas used (CDF and VaR)

3

From the estimated density $\hat{f}_h$, we define the CDF as

$$\hat{F}_h(x) = \int_{-\infty}^x \hat{f}_h(u) du.$$

Since we work on a grid $\{x_j\}$, we approximate this integral numerically (as in the notebook). With a grid step Δx ,

$$\hat{F}_h(x_j) \approx \sum_{m \leq j} \hat{f}_h(x_m) \Delta x.$$

Finally, the VaR is extracted as the point where the CDF reaches α :

$$\widehat{\text{VaR}}_\alpha = \inf\{x : \hat{F}_h(x) \geq \alpha\}.$$

We approximate the CDF implied by the KDE on a grid (numerical integration), normalize the density for numerical stability, and then locate the point where the CDF reaches α by interpolation. We keep α as a parameter (here we use 5% as the baseline level in Question A).

Code (KDE-based CDF + VaR)

```
# Risk level (easy to change)
alpha = 0.05 #5% left-tail level

# We approximate the CDF implied by the KDE by numerical integration on the grid.
# This is a simple way to extract the alpha-quantile from the non-parametric
# distribution

dx = x_grid[1] - x_grid[0]
```

³Market Risk lecture slides, M. Garcin, p. 71 (non-parametric distribution) and p. 67-68 (VaR as a quantile).

```

# Normalize the KDE to make sure it integrates to ~1 on the grid (numerical
→ stability)
kde_area = np.trapezoid(kde_vals, x_grid)
kde_pdf = kde_vals / kde_area

kde_cdf = np.cumsum(kde_pdf) * dx
kde_cdf = np.clip(kde_cdf, 0, 1) # just to avoid tiny numerical overshoots

# VaR threshold =alpha-quantile of the KDE-based distribution
VaR_nonparametric = np.interp(alpha, kde_cdf, x_grid)

print(f"Non-parametric (KDE-based) VaR threshold : {VaR_nonparametric:.6f}")

```

Output (main result of A)

```
Non-parametric (KDE-based) VaR threshold : -0.038011
```

Interpretation A 5% VaR of approximately -3.80% means that, under the distribution estimated from 2015–2016 data, about 5% of daily returns are expected to be lower than -3.80% . In other words, this threshold corresponds to a severe but plausible one-day loss, rather than an exceptional outlier.

Graphical summary (using the VaR)

At this point, the figures are not just decoration: they help us check that the VaR we extracted makes sense.

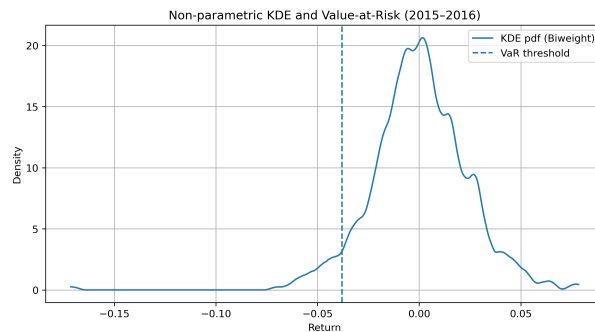


Figure 2: Non-parametric KDE and Value-at-Risk (2015–2016)

What we look at on the KDE + VaR plot. The vertical line marks the VaR threshold on the estimated density. Visually, this confirms that VaR sits on the left tail (as expected) and gives an intuitive sense of how extreme that threshold is relative to the bulk of the distribution.

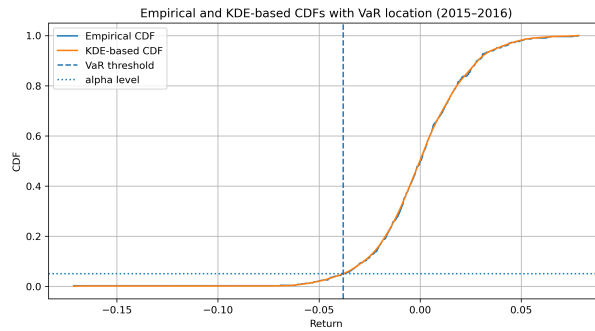


Figure 3: Empirical and KDE-based CDFs with VaR location (2015–2016)

What we look at on the CDF plot. This plot is the cleanest justification of the quantile extraction: the VaR is exactly the point where the CDF reaches α . Comparing the empirical CDF and the KDE-based CDF also reassures us that the smoothing does not completely distort the distribution.

The KDE and CDF plots confirm that the estimated VaR is correctly located in the left tail and corresponds to the chosen confidence level. The comparison with the empirical CDF also suggests that kernel smoothing does not significantly distort the underlying distribution.

A.b Out-of-sample validation (2017–2018)

Now we keep the VaR threshold learned from 2015–2016 and test it on 2017–2018. A violation occurs when the realized return is below the VaR threshold.

Formulas used (backtest)

4

Define the violation indicator:

$$I_t = \mathbf{1}_{\{r_t < \widehat{\text{VaR}}_\alpha\}}.$$

Over T out-of-sample observations, the empirical violation rate is

$$\hat{p} = \frac{1}{T} \sum_{t=1}^T I_t.$$

We compare $\hat{p}$ to α : if $\hat{p} > \alpha$, the VaR tends to underestimate risk; if $\hat{p} < \alpha$, it tends to be conservative for that period.

Code (backtest and violation rate)

```
#Question A-b: out-of-sample check (2017-2018)
returns_out = data_2017_2018["Return"].values

violations = returns_out < VaR_nonparametric
n_obs = len(returns_out)
n_viol = int(np.sum(violations))
viol_rate = n_viol / n_obs

print("Out-of-sample backtest (2017-2018)")
print(f"Observations: {n_obs}")
print(f"Violations (return < VaR): {n_viol}")
```

⁴Market Risk lecture slides, M. Garcin, p. 92.


```
print(f"Observed violation rate: {100*viol_rate:.2f}%")
print(f"Theoretical level (alpha): {100*alpha:.2f}%")
```

Output (backtest result)

```
Out-of-sample backtest (2017-2018)
Observations: 510
Violations (return < VaR): 8
Observed violation rate: 1.57%
Theoretical level (alpha): 5.00%
```

In practical terms, 8 violations over 510 trading days means the VaR threshold was crossed on about 1 to 2 days per 100, which is well below the expected 5 days per 100 at the 5% level.

The observed violation rate (1.57%) is well below the theoretical level of 5%, which indicates that the VaR estimated on 2015–2016 data is conservative when applied to the 2017–2018 period. This result is consistent with the backtesting interpretation discussed in the lectures and may reflect either a lower volatility regime or a change in market conditions out of sample.

Conclusion

Using a biweight KDE on 2015–2016 returns, we obtain a 5% one-day VaR threshold of about -3.80% (in return units). When we test this fixed threshold on 2017–2018, we observe 8 violations out of 510 days (1.57%), which suggests a conservative VaR for that out-of-sample period. Overall, this exercise illustrates how non-parametric methods can be used to estimate and validate market risk measures in a flexible way, while highlighting the importance of out-of-sample evaluation.

B Question B - Expected Shortfall

Objective and definition

In Question A, we computed a non-parametric one-day VaR using a biweight KDE on the 2015–2016 period. Here, we compute the associated Expected Shortfall (ES) at the same confidence level, using the VaR threshold obtained in Question A and the same in-sample period (2015–2016).

According to the course, the Expected Shortfall at level α is defined as the conditional expectation of the loss given that the loss is in the tail beyond the corresponding VaR level:

$$ES_{\alpha}(L) = E[L \mid L \geq \text{VaR}_{\alpha}(L)].$$

In simple terms, VaR gives a cutoff (a quantile), while ES measures the average severity of losses once we are beyond that cutoff.

In the code, we use the same tail level as in Question A: a 5% left-tail probability on returns ($\alpha = 0.05$). Equivalently, when working with losses $L = -R$, this corresponds to reporting results at the 95% confidence level.

Formulas used

⁵ We follow the course convention where returns are treated as *gains*. Let R be the one-day return (gain) and define the associated one-day *loss* as

$$L = -R.$$

This is just a sign convention, but it is practical: VaR and ES become positive numbers when we express them in losses.

From the course:

$$\text{VaR}_{\alpha}(L) = F_L^{-1}(\alpha).$$

In our case, we work with a left-tail level of 5% on returns, which corresponds to a 95% confidence level. Concretely, this is why we convert the return-based VaR from Question A into a loss threshold through

$$\text{VaR}_{0.95}(L) = -\text{VaR}_{0.05}(R).$$

For Expected Shortfall, the course gives the (equivalent) definitions:

$$ES_{\alpha}(X) = \frac{1}{1-\alpha} \int_{\alpha}^1 \text{VaR}_r(X) dr \quad \text{and} \quad ES_{\alpha}(X) = \frac{1}{1-\alpha} \int_{\alpha}^1 F_X^{-1}(r) dr.$$

In our implementation (and because we work with losses), we use the direct conditional-mean interpretation:

$$ES_{0.95}(L) = E[L \mid L \geq \text{VaR}_{0.95}(L)],$$

which we approximate by the empirical average of losses beyond the VaR threshold.

Computation (consistent with Question A)

To stay consistent with Question A, we use the same in-sample period (2015–2016) and the same VaR threshold. We define losses as negative returns and we compute ES as the empirical mean of observed losses exceeding the VaR threshold, without introducing any additional distributional assumption.

⁵Market Risk lecture slides, M. Garcin, p. 68 (VaR conventions) and p. 75 (Expected Shortfall definition).

Code

```
# Expected Shortfall computation (non-parametric, consistent with Question A)

#we work on the same in-sample period as for the VaR (2015-2016)
returns_in = data_2015_2016["Return"].values

# losses are defined as negative returns
losses_in = -returns_in

# VaR threshold at level alpha (already computed in Question A)
VaR_level = -VaR_nonparametric # convert back to losses

# Expected Shortfall: average loss beyond the VaR threshold
ES_nonparametric = losses_in[losses_in >= VaR_level].mean()

print(f"Non-parametric Expected Shortfall at 95% confidence level:
→ {ES_nonparametric:.6f}")
print(f"Non-Parametric VaR (Biweight Kernel) at 95% confidence level:
→ {VaR_level:.6f}")
```

Output

```
Non-parametric Expected Shortfall at 95% confidence level: 0.053362
Non-Parametric VaR (Biweight Kernel) at 95% confidence level: 0.038011
```

Results and comparison with VaR

We obtain:

- $\text{VaR}_{95\%} \approx 0.038011$ in losses (about 3.80%),
- $\text{ES}_{95\%} \approx 0.053362$ in losses (about 5.34%).

This is exactly the behaviour we expect: ES is higher than VaR because it averages the losses in the worst 5% days, not just the threshold that defines them. In our case, the gap is not small: once we enter the tail (beyond the VaR level), the average loss is materially larger than the VaR threshold itself.

Conclusion

VaR is a useful quantile cutoff, but Expected Shortfall provides additional information about the severity of extreme losses. In our case, the non-parametric $ES_{0.95}$ (about 5.34%) is clearly above the corresponding non-parametric $VaR_{0.95}$ (about 3.80%), which is exactly what we expect: once we are in the worst 5% tail, the average loss is materially larger than the VaR threshold itself. For a portfolio value of 100,000, this corresponds to a VaR of about 3,801 and an Expected Shortfall of about 5,336. Overall, ES is more informative for tail risk analysis because it summarizes the average magnitude of losses beyond the VaR cutoff, whereas VaR only identifies the cutoff level.

C Question C - EVT (Pickands estimator) and EVT-based VaR

In this question, we switch gear compared to Questions A–B. Instead of estimating the whole distribution (KDE) and reading a quantile, we go directly to the extreme tails with EVT. Concretely, we do two things: (i) estimate a tail index ξ using Pickands on both tails (gains vs losses), (ii) plug that ξ into the EVT VaR formula from the course to extrapolate very high confidence VaR levels (on the loss side).

C.a Pickands tail index (right tail vs left tail)

Data preparation (gains vs losses). We follow the usual EVT habit: treat tails separately. On the left tail, we work with *loss magnitudes* so that the tail variable is positive. Let R be the daily return (gain). We define the loss magnitude as

$$L = -R \quad \text{on days where } R < 0.$$

So large losses correspond to large values of L , which fits the course notation.

Code (build samples).

```
#build gain/loss samples from Natixis returns
R_nat = nat["Return"].dropna().astype(float).values

G_nat = np.sort(R_nat[R_nat > 0])          # gains strictly positive
L_nat = np.sort(-R_nat[R_nat < 0])        # losses as strictly positive magnitudes

print(f"Sample sizes -> gains: {len(G_nat)}, losses: {len(L_nat)}")
```

Output.

```
Sample sizes -> gains: 498, losses: 519
```

Formulas used (Pickands). Pickands is defined in the course as an estimator of the EVT tail parameter ξ based on order statistics.⁶ If $(X_{1:n} \leq \dots \leq X_{n:n})$ are the sorted observations of a positive tail variable, then for a choice of k :

$$\hat{\xi}_{k,n}^P = \frac{1}{\log(2)} \log \left(\frac{X_{n-k+1:n} - X_{n-2k+1:n}}{X_{n-2k+1:n} - X_{n-4k+1:n}} \right).$$

The course also stresses that k is a tuning choice: theoretically $k(n) \rightarrow \infty$ and $k(n)/n \rightarrow 0$, but in practice we try a few standard rules (that is exactly what we do in the notebook).

Code (choice of $k(n)$ and Pickands estimates).

```
for mode in ["sqrt", "log", "n_over_log"]:
    kG = k_choice(len(G_nat), mode)
    kL = k_choice(len(L_nat), mode)

    xiG = pickands_xi(G_nat, kG)
    xiL = pickands_xi(L_nat, kL)

    print(f"\nPickands tail index (xi), k(n) = {mode}")
    print(f"  Gains : k = {kG}, xi = {xiG}")
    print(f"  Losses : k = {kL}, xi = {xiL}")
```

⁶Market Risk lecture slides, M. Garcin, p. 180.

Output (main result of C.a).

```
Pickands tail index (xi), k(n) = sqrt
  Gains : k = 22, xi = 0.7195506129331686
  Losses : k = 23, xi = 0.39064816105019273

Pickands tail index (xi), k(n) = log
  Gains : k = 6, xi = 0.5772338569463368
  Losses : k = 6, xi = -0.5089715779341932

Pickands tail index (xi), k(n) = n_over_log
  Gains : k = 80, xi = -0.20197084544428853
  Losses : k = 83, xi = 0.0729484088621743
```

Interpretation (and an honest caveat). With our reference choice $k(n) = \sqrt{n}$ (so $k \approx 22$ – 23 here), we get:

$$\hat{\xi}_{\text{gains}} \approx 0.72, \quad \hat{\xi}_{\text{losses}} \approx 0.39.$$

A positive ξ is the “heavy tail” case in the course taxonomy (Fréchet-type behaviour).⁷ So at least with the baseline choice, both tails look heavy, and gains look heavier than losses.

Now the important part: the alternative $k(n)$ choices do *move* the estimate a lot (even changing sign in some cases). That is not a “bug”, it is the main practical issue of tail index estimation: Pickands is very sensitive to the threshold choice. This is why we also included a stability check in the notebook (Pickands curve as a function of k): it is a good sanity check before treating a single number as “the tail index”.

C.b EVT-based Value-at-Risk (Pickands, loss tail)

What we compute (and on which variable). Here we only focus on the loss tail (because VaR is about downside risk). We keep working with the positive loss magnitudes $L = -R$ (on days where $R < 0$), estimate ξ with Pickands, and then extrapolate VaR at very high confidence levels p .

Formulas used (EVT VaR with Pickands). The course gives the EVT VaR extrapolation formula in terms of losses and order statistics.⁸ With n observations, a tail fraction k , and $\hat{\xi}^P$ the Pickands estimate, the EVT VaR at probability p is:

$$\text{VaR}(p) = \frac{\left(\frac{k}{n(1-p)}\right)^{\hat{\xi}^P} - 1}{1 - 2^{-\hat{\xi}^P}} (X_{n-k+1:n} - X_{n-2k+1:n}) + X_{n-k+1:n}.$$

One small practical check (also mentioned in the notebook): to extrapolate beyond the empirical tail, we want $1-p < k/n$ (otherwise we are basically asking for probabilities that are not “beyond the threshold” defined by k).

Code (EVT VaR function + application).

```
def VaR_EVT_Pickands(x_sorted, k, xi, p):
    n = len(x_sorted)
    if (k <= 0) or (2*k >= n) or np.isnan(xi):
        return np.nan
```

⁷Market Risk lecture slides, M. Garcin, p. 166 (GEV families and interpretation of the sign of ξ).

⁸Market Risk lecture slides, M. Garcin, p. 187.

```

X_nk = x_sorted[n - k]           #  $X_{\{n-k+1:n\}}$ 
X_n2k = x_sorted[n - 2*k]        #  $X_{\{n-2k+1:n\}}$ 

a = k / (n * (1 - p))

if abs(xi) < 1e-8:
    coef = np.log(a) / np.log(2)
else:
    coef = (a**xi - 1) / (1 - 2**(-xi))

return coef * (X_nk - X_n2k) + X_nk

print(f"n_losses = {len(loss_tail)}")
print(f"k_ref = {k_ref}")
print(f"xi^P (loss tail) = {xi_L}")

p_list = [0.975, 0.99, 0.995, 0.999]

print("\nEVT VaR (Pickands)")
for p in p_list:
    var_loss = VaR_EVT_Pickands(loss_tail, k_ref, xi_L, p)
    var_return = -var_loss
    print(f"p={p:.3f} (tail={1-p:.3f}) -> VaR {var_return:.2%}")

```

Output (main result of C.b).

```

n_losses = 519
k_ref = 23
xi^P (loss tail) = 0.39064816105019273

EVT VaR (Pickands)
p=0.975 (tail=0.025) -> VaR -5.76%
p=0.990 (tail=0.010) -> VaR -8.51%
p=0.995 (tail=0.005) -> VaR -11.36%
p=0.999 (tail=0.001) -> VaR -21.85%

```

Interpretation These outputs are in *return units* (negative thresholds), because at the end we convert back from losses to returns. The logic is: when we go from 99% to 99.9%, we are asking for rarer and rarer tail events, so the VaR becomes more extreme (more negative). Numerically, the effect is strong here: the 99.9% EVT VaR is around -21.85% , which is far beyond what we would guess from a “smooth KDE” viewpoint.

At the same time, this is exactly where we should be cautious: EVT is powerful, but it is also *an extrapolation tool*. If the tail index estimate is unstable (C.a), the extrapolated VaR can move a lot. That is why we keep the stability check in mind and we treat these values as: “this is what the course formula gives with our Pickands calibration”, not as an unquestionable truth.

Conclusion

We applied EVT in the strict course framework: we separated gains and losses, estimated Pickands tail indices, and then computed EVT-based VaR on the loss tail. With the baseline choice $k(n) = \sqrt{n}$, we obtain a positive tail index on both sides (heavy tails), and an EVT VaR that becomes quickly very large in magnitude as p approaches 1 (e.g., about -21.85% at 99.9%). The main practical message is not “EVT gives one perfect VaR number”, but rather: EVT is

informative and sharp, *provided* the tail calibration is stable, so checking robustness in k is part of the job.

***Notebook note (reminder).** *If you want the full stability plot and the extra robustness checks (we kept them because they are genuinely useful), the notebook is the best place to look: we spent time making it readable and it goes line-by-line through the EVT calibration choices.*

D Question D - Price impact (Bouchaud's model)

Objective and the main practical constraint

In this question, we implement a simplified calibration of Bouchaud's transitory price impact model, in the spirit of the course.⁹ The honest issue with TD4 is simple: volumes are missing most of the time, so we need a specification that does not collapse the sample.

Course model and our estimable specification

The course model (Bouchaud) writes the price as the cumulative impact of past trades:

$$p_t = p_{-\infty} + \sum_{s < t} G(t-s) \varepsilon_s S_s V_s^r,$$

where $\varepsilon_s \in \{-1, +1\}$ is the trade sign, S_s is the spread, V_s is the traded volume and r is typically small (concave volume effect). We use a power-law transitory kernel:

$$G(\ell) = \frac{1}{\ell^\gamma}, \quad \ell \geq 1.$$

In the data, we work with one-step price changes aligned with the *previous* trade information:

$$\Delta p_t = p_t - p_{t-1}.$$

To make the infinite-memory sum feasible, we approximate it with a finite window of length L , and we build a lagged regressor:

$$X_t(r, \gamma) = \sum_{\ell=1}^L \ell^{-\gamma} \varepsilon_{t-\ell} S_{t-\ell} V_{t-\ell}^r, \quad \Delta p_t \approx \kappa X_t(r, \gamma) + u_t.$$

Data loading, cleaning, and alignment checks

We load TD4, rename columns, convert European decimal commas to numeric values, and align Δp_t with $(\varepsilon_{t-1}, S_{t-1}, V_{t-1})$.

Code (loading)

```
data = pd.read_csv("data/Dataset TD4.csv", sep=";")
#we display the first rows to check the structure of the data
data.head()
```

Code (cleaning: rename + numeric conversion)

```
# Rename columns for easier handling
data = data.rename(columns={
    "transaction date (i=1day=24 hours)": "time",
    "bid-ask spread": "spread",
    "volume of the transaction (if known)": "volume",
    "Sign of the transaction": "sign",
    "Price (before transaction)": "price"
})

# Convert European decimal
for col in ["time", "spread", "volume", "sign", "price"]:
    data[col] = pd.to_numeric(data[col].astype(str).str.replace(",", "."),
        ↪ errors="coerce")

data.head()
```

⁹Market Risk lecture slides, M. Garcin, p. 133 (transitory impact model) and p. 136 (parameter discussion).

Code (one-step changes + alignment diagnostics)

```
# Small but important step: we sort by time before taking differences
data = data.sort_values("time").reset_index(drop=True)

# I extract the raw series (names are the cleaned ones from the previous cell)
p = data["price"].to_numpy()
S = data["spread"].to_numpy()
V = data["volume"].to_numpy()
eps = data["sign"].to_numpy()

# One-step price changes:  $dp[i] = p[i+1] - p[i]$ 
# (so dp is aligned with the previous trade i)
dp = np.diff(p)

# Align predictors with the previous trade (same length as dp)
work = pd.DataFrame({
    "time_prev": data["time"].to_numpy()[:-1],
    "dp": dp,
    "S_prev": S[:-1],
    "V_prev": V[:-1],
    "eps_prev": eps[:-1],
})

# Very basic checks (just to see what we have before filtering)
n_trades = len(data)
n_dp = len(work)

miss_V = data["volume"].isna().sum()
miss_S = data["spread"].isna().sum()

print(f"Total trades (raw): {n_trades}")
print(f"One-step p observations: {n_dp}")
print(f"Missing volume in raw data: {miss_V} ({miss_V/n_trades:.1%})")
print(f"Missing spread in raw data: {miss_S} ({miss_S/n_trades:.1%})")

# How many observations already have the basic variables available?
valid_basic = work[["dp", "S_prev", "V_prev", "eps_prev"]].notna().all(axis=1)
print(f"Obs with (p, eps_prev, S_prev, V_prev) available: {valid_basic.sum()} /  
→ {n_dp}")

# quick check: on average, dp should be positive after eps_prev=+1 and negative after  
→ eps_prev=-1
tmp = work[["dp", "eps_prev"]].dropna()
m = tmp.groupby("eps_prev")["dp"].mean()
print(f"Mean p | eps_prev = +1: {m.get(1, np.nan)}")
print(f"Mean p | eps_prev = -1: {m.get(-1, np.nan)}")
```

Output (from our run)

```
Total trades (raw): 1001
One-step p observations: 1000
Missing volume in raw data: 864 (86.3%)
Missing spread in raw data: 0 (0.0%)
Obs with (p, eps_prev, S_prev, V_prev) available: 137 / 1000
Mean p | eps_prev = +1: 0.06333667334669339
Mean p | eps_prev = -1: -0.06119161676646709
```

The key takeaway is clear: spreads are fully observed, but volumes are missing for most trades. This is why we need a pragmatic convention in the construction of $X_t(r, \gamma)$.

Lagged regressor and missing-volume convention

If we required all lagged volumes to be observed, the usable sample would basically vanish when L is not tiny. So, in the notebook (and in this report), we keep the full trade-time sequence and use the following simple convention:

missing (or non-positive) volume $\Rightarrow$ zero contribution in the sum defining $X_t(r, \gamma)$.

Code (construction of $X_t(r, \gamma)$ + quick sanity check)

```
# Finite memory window: we only use the last L trades in the sum
L = 50

# Arrays from the aligned table "work" (built in the previous cell)
dp_arr = work["dp"].to_numpy()
S_arr = work["S_prev"].to_numpy()
V_arr = work["V_prev"].to_numpy()
eps_arr = work["eps_prev"].to_numpy()

def compute_X(eps, S, V, r, gamma, L):

    # kernel weights: 1/ell^gamma for ell = 1..L
    weights = np.arange(1, L + 1, dtype=float) ** (-gamma)

    # volume term: V~r when V is observed and > 0, else 0
    V_term = np.zeros_like(V, dtype=float)
    ok = np.isfinite(V) & (V > 0)
    V_term[ok] = V[ok] ** r

    # signed order-flow part at each trade time (before lagging)
    base = pd.Series(eps * S * V_term)

    # build X by shifting the base series (easy and readable)
    X = np.zeros(len(base), dtype=float)
    for ell in range(1, L + 1):
        X += weights[ell - 1] * base.shift(ell).fillna(0.0).to_numpy()

    return X

# quick sanity check (just to see that the construction behaves as expected)
X_check = compute_X(eps_arr, S_arr, V_arr, r=0.5, gamma=1.0, L=L)

usable = np.isfinite(dp_arr) & np.isfinite(X_check) & (np.abs(X_check) > 0)

print("Lag window L:", L)
print("Usable observations (non-zero X):", usable.sum(), "/", len(dp_arr))
print("Any NaN in X on usable rows:", np.isnan(X_check[usable]).any())
print("Share of missing volumes (V_prev):", np.isnan(V_arr).mean())
```

Output (from our run)

```
Lag window L: 50
Usable observations (non-zero X): 999 / 1000
Any NaN in X on usable rows: False
Share of missing volumes (V_prev): 0.863
```

Estimation of (κ, r, γ) (coarse grid)

For fixed (r, γ) , the model is linear in κ , so we estimate κ by OLS without intercept:

$$\hat{\kappa}(r, \gamma) = \frac{\sum_t X_t(r, \gamma) \Delta p_t}{\sum_t X_t(r, \gamma)^2}.$$

We then select (r, γ) by a simple grid search that minimizes the mean squared error:

$$\text{MSE}(r, \gamma) = \frac{1}{n} \sum_t (\Delta p_t - \hat{\kappa}(r, \gamma) X_t(r, \gamma))^2.$$

Code (coarse grid search)

```
def estimate_kappa_ols(X, Y):
    #OLS without intercept: kappa = (X'Y)/(X'X)
    denom = np.dot(X, X)
    if denom == 0:
        return np.nan
    return np.dot(X, Y) / denom

def mse(y, y_hat):
    return np.mean((y - y_hat) ** 2)

#Target: one-step price changes
Y = dp_arr

#simple grids (student choice: coarse first, refine later if needed)
r_grid = np.arange(0.0, 1.01, 0.1)
gamma_grid = np.arange(0.1, 2.01, 0.1)

best = {"mse": np.inf, "kappa": np.nan, "r": np.nan, "gamma": np.nan}

for r in r_grid:
    for gamma in gamma_grid:
        #Build X_t(r,gamma) using the aligned series (eps, spread, volume)
        X = compute_X(eps_arr, S_arr, V_arr, r=r, gamma=gamma, L=L)

        # Keep only rows where both Y and X are finite and X is not degenerate
        valid = np.isfinite(Y) & np.isfinite(X) & (np.abs(X) > 0)
        Xv, Yv = X[valid], Y[valid]

        if len(Yv) == 0:
            continue

        kappa_hat = estimate_kappa_ols(Xv, Yv)
        if np.isnan(kappa_hat):
            continue

        Y_hat = kappa_hat * Xv
        current_mse = mse(Yv, Y_hat)

        if current_mse < best["mse"]:
            best = {"mse": current_mse, "kappa": kappa_hat, "r": r, "gamma": gamma}

best
```

Output (from our run)

```
{'mse': np.float64(0.005425879131224422),  
'kappa': np.float64(-0.047615695904314595),  
'r': np.float64(0.0),  
'gamma': np.float64(2.0)}
```

Results and interpretation

On the coarse grid, the best parameters are $\hat{r} = 0.0$, $\hat{\gamma} = 2.0$, and $\hat{\kappa} \approx -0.0476$ (MSE $\approx 5.43 \times 10^{-3}$).

A simple (and honest) reading is:

- $\hat{r} = 0$ is consistent with the fact that volume information is extremely sparse, so the calibration does not really exploit volume concavity here.
- $\hat{\gamma} = 2.0$ sits on the boundary of the tested grid, so we see it more as an order of magnitude (fast decay) than as a precise estimate. A finer grid would be needed to be more confident.
- The negative $\hat{\kappa}$ depends on sign conventions (trade sign, price difference alignment) and on the missing-volume convention in the construction of X_t . So we keep the interpretation cautious: the point is more that we managed to implement a model-consistent calibration under data constraints than to claim a perfect parameter estimate.

Conclusion

Overall, this simplified Bouchaud impact calibration is consistent with the course framework, but the estimates should be interpreted carefully because (i) volume is missing for most trades in TD4 and (ii) the grid search is deliberately coarse (with a boundary optimum for γ).

If you want the full step-by-step code and intermediate checks (including the displayed tables), the notebook is the best place to look: we put quite some effort into making it readable.

E Question E - Haar wavelets, multiresolution correlations, and Hurst exponent

E.a Multiresolution correlations with Haar wavelets

We work with three FX rates: GBPEUR, SEKEUR, and CADEUR. The goal is to study how correlations change with the time scale, using a simple Haar wavelet decomposition.¹⁰

Data loading and first inspection We start by loading the dataset and checking that the columns look consistent. This step is not the “core” of the question, but it avoids silly mistakes (wrong separators, missing values, columns read as strings, etc.). The notebook contains the full inspection steps.

Code (loading)

```
df_wavelet = pd.read_csv("data/Dataset_TD5.csv", sep=";")
print(df_wavelet.head(15))
```

```
df_wavelet2 = pd.read_csv("data/Dataset_TD5_clean.csv", sep=";")
print(df_wavelet2.head())
```

Cleaning: numeric columns and mid-prices Some columns are stored with commas as decimal separators. We convert them to floats and compute the mid-price as the average between the high and low prices. This matches the statement: we do not work directly on high or low, but on a single “representative” price.

Code

```
#we convert them to floats so that we can perform basic numerical operations
cols = [
    'GBPEUR_HIGH', 'GBPEUR_LOW',
    'SEKEUR_HIGH', 'SEKEUR_LOW',
    'CADEUR_HIGH', 'CADEUR_LOW'
]

for c in cols:
    df_wavelet2[c] = (df_wavelet2[c].astype(str).str.replace(',', '.'),
        ↪ regex=False).astype(float) )
```

```
# We compute the mid-price as the average between the high and the low prices
df_wavelet2['GBPEUR_avg'] = (df_wavelet2['GBPEUR_HIGH'] + df_wavelet2['GBPEUR_LOW'])
    ↪ / 2
df_wavelet2['SEKEUR_avg'] = (df_wavelet2['SEKEUR_HIGH'] + df_wavelet2['SEKEUR_LOW'])
    ↪ / 2
df_wavelet2['CADEUR_avg'] = (df_wavelet2['CADEUR_HIGH'] + df_wavelet2['CADEUR_LOW'])
    ↪ / 2
```

Returns at the finest scale We compute returns at the finest available frequency (15 minutes). This is the “raw” level where microstructure and asynchronicity effects are usually the strongest.

¹⁰Market Risk lecture slides, M. Garcin: Haar wavelet and multiresolution analysis (page 291).

Code

```
# returns are computed at the finest sampling frequency (15 minutes)
df_wavelet2['GBPEUR_returns'] = df_wavelet2['GBPEUR_avg'].pct_change().dropna()
df_wavelet2['SEKEUR_returns'] = df_wavelet2['SEKEUR_avg'].pct_change().dropna()
df_wavelet2['CADEUR_returns'] = df_wavelet2['CADEUR_avg'].pct_change().dropna()
```

```
df_wavelet2 = df_wavelet2.dropna()
```

Haar decomposition and correlations by level We apply a Haar decomposition with 3 levels and compute correlations on the detail coefficients at each level. The idea is simple: each level corresponds to a coarser “view” of the signal, so we can check whether correlations become more visible when we move away from the highest frequency.

Since the original sampling is 15 minutes, the decomposition naturally moves to coarser horizons: Level 1 corresponds roughly to 30 minutes, Level 2 to 60 minutes, and Level 3 to 120 minutes (each level aggregates information by a factor 2).

Formulas used

¹¹ We keep the math part very light here, but we still want to be explicit about what the code is doing.

Haar one-step decomposition. Given a signal $(x_0, x_1, x_2, \dots)$, the Haar transform builds, for each pair, an approximation (low frequency) and a detail (high frequency):

$$a_k = \frac{x_{2k} + x_{2k+1}}{2}, \quad d_k = \frac{x_{2k} - x_{2k+1}}{2}.$$

In our implementation, we iterate this idea over several levels. Each level gives us a “coarser view” of the returns, and we compute correlations on the detail coefficients of each level.

A small technical note from the course: wavelet covariances have a scale-dependent amplitude, but working with correlations (as we do here) already normalizes by the variances and makes results comparable across levels.

Correlation. For two series X and Y , the correlation is

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}.$$

Here, we compute these correlations on the wavelet detail coefficients at each level, which is why the numbers can change from Level 1 to Level 3.

Code

```
returns = df_wavelet2[['GBPEUR_returns', 'SEKEUR_returns', 'CADEUR_returns']].values

# Apply a simple Haar-wavelet decomposition to study correlations at multiple scales
def haar_decomposition(signal, levels):
    coeffs = []
    current_data = signal.copy()
    for level in range(levels):
        # Ensure even length
        current_data = current_data[:len(current_data)//2*2]
        # Approximations (low frequency) and details (high frequency)
```

¹¹Market Risk lecture slides, M. Garcin: Haar wavelet and multiresolution analysis (page 291).

```

        approx = 0.5 * (current_data[0::2] + current_data[1::2])
        detail = 0.5 * (current_data[0::2] - current_data[1::2])
        coeffs.append(detail)
        current_data = approx
    return coeffs

levels = 3 # We look at 3 levels of decomposition
gbp_coeffs = haar_decomposition(returns[:, 0], levels)
sek_coeffs = haar_decomposition(returns[:, 1], levels)
cad_coeffs = haar_decomposition(returns[:, 2], levels)

# Compute correlations at each level
correlations = {}
for i in range(levels):
    level_name = f"Level {i+1}"
    corr_matrix = np.corrcoef(np.vstack([gbp_coeffs[i], sek_coeffs[i],
    ↪ cad_coeffs[i]]))
    correlations[level_name] = {
        'GBPEUR-SEKEUR': corr_matrix[0, 1],
        'GBPEUR-CADEUR': corr_matrix[0, 2],
        'SEKEUR-CADEUR': corr_matrix[1, 2]
    }

# Display results
for level, corr in correlations.items():
    print(f"Correlations at {level}: {corr}")

```

Output

```

Correlations at Level 1: {'GBPEUR-SEKEUR': 0.16012522885880082, 'GBPEUR-CADEUR':
↪ 0.21821619027674313, 'SEKEUR-CADEUR': 0.17423835750647093}
Correlations at Level 2: {'GBPEUR-SEKEUR': 0.13998285009850261, 'GBPEUR-CADEUR':
↪ 0.32756356206770765, 'SEKEUR-CADEUR': 0.2537114609268602}
Correlations at Level 3: {'GBPEUR-SEKEUR': 0.29973610869635015, 'GBPEUR-CADEUR':
↪ 0.2259544430503508, 'SEKEUR-CADEUR': 0.15999084392100796}

```

Interpretation We do see that correlations depend on the scale, but it is not a perfectly monotone story for every pair. For example, GBPEUR–SEKEUR becomes noticeably higher at Level 3 (about 0.30), while GBPEUR–CADEUR peaks at Level 2 (about 0.33). So the message is: correlations are not “one number” here; they evolve with the resolution. This is consistent with the general idea behind the Epps effect: at the highest frequency, correlations can look weaker or noisier, and they tend to stabilize when we move to coarser scales.¹²

E.b Hurst exponent and volatility scaling

We now estimate the Hurst exponent for each FX return series and use it to scale volatility.

Hurst estimation We compute H for each series with the method implemented in the notebook. We keep the interpretation simple: $H \approx 0.5$ is close to the Brownian scaling intuition, while values above 0.5 suggest persistence.

¹²Market Risk lecture slides, M. Garcin: Multiresolution covariance and Epps effect (p. 293).

Formulas used

We follow the exact logic implemented in the notebook. The goal is not to introduce a new estimator, but to clearly write what the code computes.¹³

Hurst scaling idea. The scaling rule discussed in the course is that volatility behaves like a power law in time:

$$\sigma(\Delta t) \propto (\Delta t)^H.$$

A classical Brownian motion corresponds to $H = \frac{1}{2}$. In the course language: if $H = \frac{1}{2}$, we recover the usual $\sqrt{\Delta t}$ scaling; if $H > \frac{1}{2}$, increments are persistent; if $H < \frac{1}{2}$, increments are anti-persistent.

In our case, we estimate H from data (with the notebook estimator below) and then annualize volatility using N^H , where $N = 252 \times 10 \times 4$ is the approximate number of 15-minute intervals in one trading year (252 trading days, about 10 trading hours per day, and 4 intervals per hour).

Estimator used in the notebook. Let $(X_t)_{t=1,\dots,T}$ be the return series and $\bar{X}$ its empirical mean. With $k = 2$, the code computes:

$$M_2 = \frac{1}{T-1} \sum_{t=2}^T |X_t - \bar{X}|^k, \quad M'_2 = \frac{1}{T/2-1} \sum_{t=2}^{T/2} |X_t + X_{t+T/2} - 2\bar{X}|^k,$$

and then

$$H = \frac{1}{k} \frac{\log\left(\frac{M'_2}{2M_2}\right)}{\log 2}.$$

Annualization used in the notebook. Once we have H , the annualized volatility in the code is

$$\sigma_{\text{ann}} = \sigma_{15\text{min}} \cdot N^H, \quad N = 252 \times 10 \times 4.$$

As a quick check: if $H = \frac{1}{2}$, this reduces to $\sigma_{\text{ann}} = \sigma_{15\text{min}} \sqrt{N}$.

14

Code

```
def Hurst_Exponent(returns):
    k = 2 # the moment of order 2 is used for volatility estimation

    #Step 1: compute mean
    mean = sum(returns)/len(returns)

    # Step 2: compute M2(1)
    M_2 = 0
    for i in range(1, T):
        M_2 += abs(returns[i] - mean)**k
    M_2 = M_2 / (T - 1)

    # Step 3: compute M2'(T)
    M_2_prime = 0
    for i in range(1, T // 2):
        M_2_prime += abs(returns[i] + returns[i + T // 2] - 2 * mean)**k
    M_2_prime = M_2_prime / (T/2 - 1)
```

¹³Market Risk lecture slides, M. Garcin: Hurst exponent and volatility scaling (page 210).

¹⁴Market Risk lecture slides, M. Garcin: two-scale estimator of H (p. 252).


```

# Step 4: compute Hurst exponent
ratio = M_2_prime / M_2

H = (1 / k) * np.log(ratio / 2) / np.log(2)

return H

#Creating a list and storing results for each currency
X_GBPEUR = df_wavelet2["GBPEUR_returns"].dropna().values.astype(float)
X_SEKEUR = df_wavelet2["SEKEUR_returns"].dropna().values.astype(float)
X_CADEUR = df_wavelet2["CADEUR_returns"].dropna().values.astype(float)

T = min(len(X_GBPEUR), len(X_SEKEUR), len(X_CADEUR)) # Common length

X_GBPEUR = X_GBPEUR[:T]
X_SEKEUR = X_SEKEUR[:T]
X_CADEUR = X_CADEUR[:T]

Hurst_GBPEUR = Hurst_Exponent(X_GBPEUR)
Hurst_SEKEUR = Hurst_Exponent(X_SEKEUR)
Hurst_CADEUR = Hurst_Exponent(X_CADEUR)

print("Hurst exponent fo GBPEUR:",Hurst_GBPEUR)
print("Hurst exponent fo SEKEUR:",Hurst_SEKEUR)
print("Hurst exponent fo CADEUR:",Hurst_CADEUR)

```

Output

```

Hurst exponent for GBPEUR: 0.686860136564295
Hurst exponent for SEKEUR: 0.6810236267055336
Hurst exponent for CADEUR: 0.6810811380086749

```

Interpretation All three estimates are clearly above 0.5 (around 0.68). We do not over-interpret it, but it suggests that scaling may deviate from the classical $\sqrt{T}$ rule, which is exactly why the course discusses volatility scaling with H .

Annualized volatility with Hurst scaling We compute annualized volatility consistently with the 15-minute sampling frequency, using the scaling N^H (as implemented in the notebook), where $N = 252 \times 10 \times 4$. This gives a volatility measure that is consistent with the Hurst-based scaling behaviour.

Code

```

def calculate_annualized_volatility(returns, hurst_exponent):
    #We compute the mean
    mean_return = sum(returns)/len(returns)

    #We compute the standard deviation
    std_return = np.sqrt(sum((r - mean_return)**2 for r in returns)/(len(returns)-1))

    # We now compute annualized volatility using Hurst exponent returns
    #There is 252 of trading days in a year
    #The time scaling is  $252^H$ 
    annualized_vol = std_return * (252 * 10 * 4) ** hurst_exponent

    return annualized_vol

```

```

# Annualized volatility computation
annually_vol_GBPEUR = calculate_annualized_volatility(
    df_wavelet2["GBPEUR_returns"].dropna().values.astype(float),
    Hurst_GBPEUR
)
print(f"Annualized volatility for GBPEUR: {annually_vol_GBPEUR:.6f}")

annually_vol_SEKEUR = calculate_annualized_volatility(
    df_wavelet2["SEKEUR_returns"].dropna().values.astype(float),
    Hurst_SEKEUR
)
print(f"Annualized volatility for SEKEUR: {annually_vol_SEKEUR:.6f}")

annually_vol_CADEUR = calculate_annualized_volatility(
    df_wavelet2["CADEUR_returns"].dropna().values.astype(float),
    Hurst_CADEUR
)
print(f"Annualized volatility for CADEUR: {annually_vol_CADEUR:.6f}")

```

Output

```

Annualized volatility for GBPEUR: 0.350331
Annualized volatility for SEKEUR: 0.174243
Annualized volatility for CADEUR: 0.269808

```

Interpretation The three annualized volatilities are quite different across pairs: GBPEUR is the highest here (about 0.35), SEKEUR is the lowest (about 0.17), and CADEUR sits in between (about 0.27). The important point is not just the ranking, but the logic: once H is above 0.5, scaling can amplify volatility relative to the standard square-root rule, which is exactly what this part is designed to illustrate.

Conclusion

Wavelet correlations show that dependence between FX returns is scale-dependent, and our results are consistent with a partial Epps effect: correlations are generally more stable at coarser levels, even if the pattern is not perfectly monotone across all pairs. The Hurst exponent estimates (around 0.68) suggest persistence-like scaling behaviour in this dataset, and using the 252^H scaling leads to annualized volatilities that differ meaningfully across the three FX pairs.

General conclusion

This project helped us connect several market risk tools to real datasets, with one main idea in mind: results are only meaningful if we understand the assumptions behind them (distributional choices, tail modelling, and time scale) and if we keep an eye on robustness.

Main takeaways by question

- **A – Non-parametric VaR:** Using a biweight KDE on 2015–2016 Natixis returns, we obtained a 5% one-day VaR threshold around -3.80% . The 2017–2018 backtest produced fewer violations than 5% (1.57%), so this VaR looks conservative on that out-of-sample period.
- **B – Expected Shortfall:** ES is higher than VaR (about 5.34% vs 3.80% in losses). The key point is simple: VaR is a cutoff, ES describes the average severity once we are already in the tail.
- **C - EVT (Pickands):** Using Pickands in the EVT/GEV framework, we estimated tail behaviour separately for extreme gains and losses. With the baseline choice $k(n) = \sqrt{n}$, the EVT-based loss VaR becomes quickly more extreme as p approaches 1 (e.g., around -21.85% at 99.9%). The practical takeaway is that these numbers are very sensitive to the choice of k , so stability checks are crucial before trusting EVT extrapolations.
- **D – Bouchaud price impact:** On TD4, volumes are missing most of the time, so we used a pragmatic specification and a coarse grid search. The best coarse-grid fit is obtained at $\hat{r} = 0.0$, $\hat{\gamma} = 2.0$ (boundary of the grid) and $\hat{\kappa} \approx -0.0476$. The model runs end-to-end and is course-consistent, but parameter values should be interpreted cautiously given missing volumes and the boundary optimum.
- **E – Wavelets and scaling:** Correlations depend on the time scale (partial Epps effect), and Hurst exponents around 0.68 suggest volatility scaling different from the standard square-root rule. We then annualize volatility by combining a base volatility at the relevant time step with the Hurst scaling (i.e., scaling with a power H), which is exactly the point of the course question.

Notebook disclaimer

This report is meant to be readable and self-contained, but the notebook is where everything is fully traceable. We spent time making it clean and easy to follow (structured by question, with intermediate checks and comments next to the code). If you want full details or to reproduce results step by step, the notebook is the best place to look.